

12d — CQRS (Command Query Responsibility Segregation)

Key Concepts

- **CQRS** = Separate the code that writes data from the code that reads data
 - Commands = write operations (Create, Update, Delete)
 - Queries = read operations (fetch, project)
 - Each side has its own model, handlers, and optionally its own database
 - In .NET, **MediatR** is the standard library for implementing CQRS
-

How It Works

```
Client
├── Command → CommandHandler → Write DB (normalized, consistent)
└── Query  → QueryHandler  → Read DB (denormalized, fast)
```

Command Side

- Handles writes — `PlaceOrderCommand`, `CancelOrderCommand`
- Enforces business rules and validation
- Returns only **success or failure** — **never data**

Query Side

- Handles reads — `GetOrderByIdQuery`, `GetOrderHistoryQuery`
 - Returns DTOs shaped for what the UI needs
 - No business logic — just fetch and return
 - Can read from a separate read-optimized database
-

Separate Database Pattern (Advanced)

```
Write side → Normalized SQL DB (consistency first)
              ↓ Domain Events sync changes
Read side  → Denormalized Read DB (speed first)
              (flat pre-joined tables, Redis, Elasticsearch)
```

- When something is written, a Domain Event fires
 - Event handler updates the flat read table
 - Queries hit the flat table — no joins, instant response
-

MediatR in .NET

```
// Command object
public class PlaceOrderCommand : IRequest<bool> {
    public Guid CustomerId { get; set; }
}

// Handler
public class PlaceOrderHandler : IRequestHandler<PlaceOrderCommand, bool> {
    public Task<bool> Handle(PlaceOrderCommand cmd, CancellationToken ct) {
        // business logic here
        return Task.FromResult(true);
    }
}

// Usage - caller is decoupled from handler
await _mediator.Send(new PlaceOrderCommand { CustomerId = id });
```

- Commands/Queries implement `IRequest`
- Handlers implement `IRequestHandler<TRequest, TResponse>`
- `mediator.Send()` dispatches to correct handler via DI
- Domain Events use `INotification` + `INotificationHandler`

Command vs Query — Key Rules

	Command	Query
Purpose	Write / modify state	Read / return data
Returns	Success or failure only	Data (DTO)
Business logic	Yes — validation, rules	No
Database	Write DB (normalized)	Read DB (denormalized)

When to Use CQRS

Use when	Avoid when
Read/write loads are very different	Simple CRUD app — overkill
Reads need complex projections	Small team — adds complexity
Need to scale reads/writes independently	No performance problem to solve
Domain has complex business rules	

Industry Examples

Company	CQRS Application
Microsoft	MediatR — standard across .NET enterprise apps
Uber	Trip writes (strict consistency) vs trip reads (fast dashboard)
LinkedIn	Feed writes via command handlers; feed reads from pre-built projections
Amazon	Order placement (command) vs order history page (query from read replica)

Interview Questions

Q1: What does CQRS stand for and what is the core idea?

Command Query Responsibility Segregation — separate code that writes data from code that reads data.

Q2: What problem does CQRS solve?

Writes need strict business rules and consistency. Reads need speed and flexible projections. One model serving both leads to compromise on both sides.

Q3: What do commands return vs queries?

Commands return only success or failure — never data. Queries return data — never modify state.

Q4: How does the separate database approach work?

Write side uses normalized SQL for consistency. Domain Events sync changes to a denormalized read DB with pre-built flat tables. Queries hit the read side — no joins, instant response.

Q5: How does MediatR implement CQRS in .NET?

Commands and queries implement IRequest. Each has a dedicated handler. mediator.Send() dispatches to the correct handler via DI — caller and handler are fully decoupled.